

데이터베이스, 여기서부터 시작하세요

인턴 · 초급 개발자를 위한 DB 입문 — 종류 · 핵심 개념 · 최신 트렌드

온보딩 참고자료 · 2026

※ 개념 먼저, 제품 이름은 그 다음

데이터베이스(DB)는 데이터를 **안전하게 저장하고, 빠르게 찾고, 정확하게 바꾸기** 위한 시스템입니다. 제품 이름을 외우는 것보다 '왜 이런 종류가 나뉘고, 무엇을 보장하려 하는지'를 이해하는 것이 훨씬 오래 남습니다. 이 자료는 그 큰 그림을 잡는 것을 목표로 합니다.

1 가장 큰 갈래 — SQL(관계형) vs NoSQL

DB를 이해하는 빠대는 '정합성이나, 확장성이나'의 트레이드오프입니다. 관계형(SQL)은 정확함과 규칙을, NoSQL은 유연함과 확장을 우선합니다.

구분	SQL (관계형 · RDBMS)	NoSQL
데이터 구조	행·열로 이뤄진 표(테이블), 정해진 스키마	유연한 스키마(문서·키값 등 다양)
강점	정확성·복잡한 조회(JOIN)·트랜잭션(ACID)	대용량·빠른 쓰기·수평 확장
확장 방식	주로 수직 확장(장비 성능 ↑)	주로 수평 확장(장비 대수 ↑)
대표 제품	Oracle, MySQL, MariaDB, MSSQL, PostgreSQL	MongoDB, Redis, Cassandra, Neo4j
잘 맞는 곳	금융·결제·업무시스템 등 정확함이 필수	로그·캐시·대규모 콘텐츠·실시간

자주 하는 착각: PostgreSQL은 NoSQL이 아닙니다

PostgreSQL(포스트그레)은 Oracle·MySQL과 같은 정통 관계형(SQL) DB입니다. 오히려 표준 SQL 지원이 가장 충실한 축이예요. NoSQL로 오해하기 쉬우니 분류할 때 주의하세요. 참고로 NoSQL도 한 종류가 아니라 여러 유형으로 나뉩니다 — 문서형(MongoDB), 키-값형(Redis), 컬럼형(Cassandra), 그래프형(Neo4j). 'MongoDB는 문서를 통째로 저장, Redis는 초고속 키-값 저장'처럼 성격이 다릅니다.

2 인메모리 DB — 디스크가 아닌 '메모리'에 담는다

보통의 DB는 데이터를 디스크에 저장하지만, **인메모리 DB는 데이터를 메모리(RAM)에 올려두고 처리**합니다. 디스크를 거치지 않으니 읽기·쓰기가 압도적으로 빠릅니다. 대표 주자는 Redis와 Memcached예요.

왜 쓰나 (강점) — 속도. 마이크로초 단위 응답이 필요한 곳에 씁니다. 캐시(자주 쓰는 결과 임시 저장), 세션 저장, 실시간 순위표(랭킹), 조회수·좋아요 카운팅, 요청 제한(rate limit), 대기열·발행/구독 등.

주의할 점 — 메모리는 비싸고 용량이 작습니다. 그리고 전원이 꺼지면 데이터가 사라질 수 있어요(휘발성). 그래서 '원본은 관계형 DB에, 빠르게 자주 읽는 것만 인메모리에' 두는 조합이 흔합니다.

한 가지 오해 — Redis는 스냅샷·로그로 디스크에 데이터를 남기는 영속화(persistence) 옵션도 있어 완전히 휘발성만은 아닙니다. 다만 기본 성격은 '빠른 임시 계층'으로 이해하면 됩니다.

그림으로 기억하기

사용자 → (먼저) Redis 캐시 확인 → 있으면 즉시 응답 / 없으면 → 관계형 DB 조회 후 결과를 Redis에 저장 → 다음엔 빠르게 응답. 이 '캐시 히트/미스' 패턴이 인메모리 DB의 가장 흔한 쓰임새입니다.

3 조금 개발자가 꼭 알면 좋은 핵심 개념

제품 이름보다 이 개념들이 실무에서 훨씬 자주 쓰입니다.

기본키 · 외래키 (PK · FK) — 각 행을 유일하게 구분하는 값이 기본키, 다른 테이블을 참조하는 연결고리가 외래키. 관계형 DB에서 데이터를 연결하는 뼈대.

트랜잭션 · ACID — 여러 작업을 '전부 성공 아니면 전부 취소(commit/rollback)'로 묶는 것. 계좌 이체처럼 중간에 깨지면 안 되는 처리의 핵심. 정확성이 생명인 곳에서 필수.

인덱스 (Index) — 책의 색인처럼 조회를 빠르게 해주는 장치. 단, 무분별하게 많이 걸면 쓰기 속도가 느려지고 저장공간도 늘어남 — 필요한 곳에만.

정규화 vs 비정규화 — 중복을 줄여 정합성을 지키는 게 정규화, 조회 성능을 위해 일부러 중복을 허용하는 게 비정규화. 상황에 따라 균형을 잡는 문제.

N+1 쿼리 문제 — ORM 쓸 때 목록 1번 + 각 항목마다 1번씩 쿼리가 나가 성능이 무너지는 대표적 실수. 조인이나 일괄 조회로 해결.

SQL 인젝션 · 파라미터 바인딩 — 사용자 입력을 문자열로 이어붙여 쿼리를 만들면 해킹당함. 반드시 파라미터 바인딩(?, :param)으로 처리. 공공/보안 환경에서 특히 중요.

커넥션 풀 — DB 연결을 미리 만들어 재사용하는 것. 매 요청마다 새로 연결하면 느리고 금방 한계에 부딪힘.

백업 · 복구, 실행계획 — 장애·실수에 대비한 백업은 기본기. 느린 쿼리는 EXPLAIN으로 실행계획을 보고 인덱스를 점검. 그리고 습관적인 SELECT * 는 피하기.

4 규모가 커지면 — 확장 · 샤딩 · 클러스터링 · MSA

수직 확장 vs 수평 확장 — 한 대에 CPU·RAM을 더 얹는 게 수직 확장, 장비 대수를 늘리는 게 수평 확장. 관계형 DB는 전통적으로 수직 확장에 강했고, 결국 한 대의 한계에 부딪히면 수평 확장이 필요해집니다.

복제 (Replication) — 같은 데이터를 여러 서버에 복사해 두는 것. 읽기 부하 분산과 장애 대비에 씬(주 서버 + 복제 서버).

샤딩 (Sharding) — 데이터를 기준(예: 지역·사용자 ID)에 따라 여러 서버에 쪼개 저장. 한 서버에 몰리는 병목을 막는 대표적 수평 확장 방식.

클러스터링 (Clustering) — 여러 DB 노드를 묶어 하나처럼 동작시키는 것. 장애가 나도 다른 노드가 이어받아 서비스가 죽지 않게 함.

MSA와 DB — 마이크로서비스 구조에서는 서비스마다 자신의 DB를 갖는(Database per Service) 경우가 많습니다. 서비스가 늘고 트래픽이 커지면서 자연스럽게 샤딩·클러스터링 수요가 생깁니다.

5 요즘 뜨는 것 — NewSQL과 벡터 DB

NewSQL (분산 SQL, Distributed SQL) — SQL의 정확함(ACID)은 유지하면서 NoSQL 같은 수평 확장을 함께 얻으려는 새로운 관계형 DB 갈래. 개발자에게 평범한 SQL DB처럼 보이지만, 내부적으로 데이터를 여러 서버에 자동 샤딩·복제합니다. 앞의 MSA·샤딩 흐름과 정확히 맞물리는 트렌드예요. 대표: CockroachDB, TiDB, YugabyteDB, Google Spanner.

벡터 DB (Vector Database) — AI 때문에 급부상. 문장·이미지를 숫자 벡터(임베딩)로 바꿔 저장하고 '의미가 비슷한 것'을 찾아줍니다. 추천, 의미 검색, 그리고 LLM의 RAG(검색 증강 생성)에 필수. 대표: Pinecone, Milvus, Qdrant, Chroma. 이미 PostgreSQL을 쓴다면 pgvector 확장으로 관계형 데이터와 벡터를 한 곳에서 다룰 수 있어, 많은 팀에 가장 간단한

선택지로 꼽힙니다.

그 외 특수 목적 DB — 시계열 DB(InfluxDB·TimescaleDB, 센서·모니터링 데이터에 강함), 그래프 DB(Neo4j, 관계망 분석에 강함), 여러 모델을 한 엔진에서 다루는 멀티모델 DB도 있습니다. 이름 정도만 알아두면 충분.

6 한눈 정리 — 무엇을 언제 쓰나 & 첫걸음 체크리스트

이럴 땐	이런 DB
정확함·복잡한 조회·트랜잭션이 중요 (결제·업무)	관계형 SQL — PostgreSQL, MySQL, Oracle 등
스키마가 자주 바뀌거나 대용량 문서	문서형 NoSQL — MongoDB
초고속 캐시·세션·랭킹·실시간 카운팅	인메모리 — Redis
SQL 정확함 + 대규모 수평 확장 동시에	NewSQL — CockroachDB, TiDB
AI 의미 검색·추천·RAG	벡터 DB — Pinecone, Milvus, 또는 pgvector

초급 개발자 첫걸음 체크리스트

① SELECT · INSERT · UPDATE · DELETE 기본 SQL을 직접 쳐본다 ② JOIN으로 두 테이블을 엮어본다 ③ 트랜잭션으로 commit / rollback을 실험해본다 ④ 인덱스를 걸기 전·후 조회 속도를 비교해본다 ⑤ 파라미터 바인딩으로 안전하게 쿼리를 만든다 ⑥ EXPLAIN으로 실행계획을 읽어본다. 이 여섯 가지를 손으로 해보면 대부분의 실무 감각이 잡힙니다.

마무리 — DB의 세계는 결국 '정확함 ↔ 확장성' 사이에서 무엇을 우선할지 고르는 문제입니다. 제품 이름은 계속 바뀌지만 이 관점은 변하지 않아요. 하나를 완벽히 아는 것보다, 상황에 맞는 도구를 고를 줄 아는 감각이 더 중요합니다.